

**SEGONA PART DE**  
**TEORIA PAR**  
**(TEMES 4 I 5)**

## **ÍNDEX**

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| <b>1. UMA, NUMA i “First Touch”</b>                                 | <b>3</b>  |
| 1.1 Traduït als exàmens                                             | 3         |
| <b>2. Protocols de Coherència de Cache</b>                          | <b>4</b>  |
| 2.1. Protocol Snooping (MSI) per a coherència Intra-node (UMA)      | 4         |
| 2.2. Protocol de Directori (MSU) per a coherència Inter-node (NUMA) | 4         |
| 2.3. Traduït als exàmens                                            | 4         |
| <b>3. Tràfic de Coherència (True Sharing vs. False Sharing)</b>     | <b>6</b>  |
| 3.1. True Sharing                                                   | 6         |
| 3.2. False Sharing                                                  | 6         |
| 3.3. Traduït a exàmens                                              | 6         |
| <b>4. Suport Hardware per a la Sincronització</b>                   | <b>8</b>  |
| 4.1. Traduït als exàmens                                            | 9         |
| <b>5. Implementació manual del Load Balancing</b>                   | <b>10</b> |
| 5.1. Distribució geomètrica per Blocs (Block Data Decomposition)    | 10        |
| 5.2. Distribució Cíclica (Cyclic Data Decomposition)                | 11        |
| 5.3. Distribució Bloc-Cíclica (Block-Cyclic Data Decomposition)     | 11        |
| <b>6. Quina Descomposició Geomètrica de Dades escollir</b>          | <b>12</b> |
| <b>7. Ajust de la Línia de Cache per a evitar False Sharing</b>     | <b>13</b> |
| <b>8. Suport Hardware per a la Sincronització</b>                   | <b>14</b> |
| 8.1. Output Data Decomposition                                      | 14        |
| 8.2. Input Data Decomposition                                       | 14        |

# 1. UMA, NUMA i “First Touch”

Les architectures multiprocessador de memòria compartida es divideixen principalment en dues categories:

- **UMA (uniform memory acces):** existeix un únic espai de direccions compartit i el temps d'accés a la memòria és uniforme per a tots els processadors.
- **NUMA (non-uniform memory acces):** l'espai de direccions segueix sent el mateix, però ara la memòria física està distribuïda en diferent nodes. El temps d'accés és ràpid si el processador accedeix a la seva memòria local (el seu propi node) i més lent si ha d'accedir a la memòria remota d'un altre node a través de la red d'interconnexió.

En els sistemes NUMA, el sistema operatiu ha de decidir en quin node físic ubicar una pàgina de memòria. Per fer això utilitza la política *First Touch*: el node que llegeix o escriu per primer cop en una pàgina de memòria es converteix en el *node home* de totes les línies de memòria d'aquesta pàgina.

## 1.1 Traduït als exàmens

En els exercicis d'examen es sol plantejar un codi on es declaren vectors o matrius i s'executen bucles sobre ells. L'objectiu és identificar què thread (i per tant que processador NUMA) fa el First Touch de les dades per a determinar on estaran allotjats físicament.

Hi ha dos escenaris possibles que poden confondre:

- **Cas 1, inicialització seqüencial:** l'error més comú en programació paral·lela (i la trampa més habitual en els exàmens) es inicialitzar les dades seqüencialment abans de processar-ho en paral·lel:
  - Exemple examen 2022/23-Q1: et donen un vector *count*, i abans de la regió paral·lela (`#pragma omp parallel`) hi ha un bucle `for(int k = 0; k < NUM_THREADS; ++k) count[k] = 0;` que l'inicialitza. A l'executar-se fora de la regió paral·lela aquest bucle l'executa el processador 0. Per la política *First Touch*, tot el vector *count* queda allotjat al NUMAnode 0. Això genera un possible coll d'ampolla posterior perquè tota la resta de threads hauran d'accedir a la memòria d'aquell node.
  - Exemple examen 2024/25-Q2: et posen la inicialització dins de la regió paral·lela, però protegida amb un `if (myid == 0)`. Encara que estiguem dintre de la regió paral·lela el thread 0 és l'únic que inicialitza el vector *x* amb un `for`. El resultat és exactament el mateix que abans.
- **Cas 2, inicialització paral·lela (comportament ideal):** per a distribuir correctament les dades en un sistema NUMA, la inicialització s'ha de fer en paral·lel, assegurant-se que cada thread inicialitza la part de dades que utilitzarà:
  - Exemple examen 2022/23-Q1: es declara la matriu *hist* i l'enunciat especifica clarament que “no hi ha accés a *hist* abans de la regió paral·lela”. Dins del bloc paral·lel, cada thread inicialitza la seva posició corresponent: `hist[i]=foo()`
  - L'anàlisi a fer: en aquest sistema hi ha threads del 0 al 3 en el NUMAnode 0, i del 4 al 7 en el NUMAnode 1. Com els threads del 0 al 3 són els primers en tocar les posicions de *hist*, la primera línia de memòria s'allotja al NUMAnode

0. Com els threads del 4 al 7 són els primers en tocar les següents posicions, la segona línia de memòria de la matriu s'allotja al NUMAnode 1.

En resum, quan preguntin quin és el Home NUMAnode d'una estructura de dades, cal buscar en el codi la primera vegada que es fa un accés (read o write) a aquella variable. Identifica què thread executa aquella línia, troba en quin node/processador està aquell thread, i aquest serà el home node.

## **2. Protocols de Coherència de Cache**

El problema de la coherència de cache sorgeix en els sistemes multiprocessador perquè múltiples processadors poden tindre còpies de la mateixa variables en les seves cache privades. Si un d'ells la modifica, els demés han d'enterar-se per a no utilitzar dades obsoletes.

### **2.1. Protocol Snooping (MSI) per a coherència Intra-node (UMA)**

Dins d'un mateix node (on diversos processadors comparteixen bus) s'utilitza un mecanisme d'Snooping (cotilleo), on tots els controladors de cache escolten les transaccions del bus. Utilitza el protocol d'invalidació MSI, que associa 2 bits a cada línia de cache per a codificar 3 estats:

- **M** (modified): la línia es bruta (modificada) i aquest processador és l'únic propietari.
- **S** (shared): la línia es neta i pot estar replicada en vàries cache.
- **I** (invalid): la línia no té dades vàlides (o no hi són a cache).

Events i transaccions clau en Snooping:

- Events de CPU: PrRd (read) i PrWr (write).
- Events de Bus:
  - BusRd: demana una copia sense intenció de modificar (pas de I a S)
  - BusRdX: demana una copia sense intenció de modificar i invalida a la resta (pas de I a M)
  - BusUpgr: demana permís per a modificar una línia que ja té (pas de S a M, invalida a la resta)
  - Flush: llença les dades al bus quan un altre processador els necessita (sempre passa quan algú demana una línia que estava en estat M).

### **2.2. Protocol de Directori (MSU) per a coherència Inter-node (NUMA)**

Com emetre transaccions pel bus a tots els nodes no escala (generaria massa tràfic), entre els nodes NUMA s'usa un directori en memòria. Cada node té un directori que busca on són les línies de la seva memòria. Cada entrada al directori associa:

- 2 bits d'estat (MSU):
  - **M** (modified): la línia es modificada en el sistema (només un node la té).
  - **S** (shared): un o més nodes tenen còpies netes.
  - **U** (uncached): cap cache en el sistema té la línia.
- Bits de presència (sharers lists): s'usa 1 bit per a cada node NUMA del sistema. Si el bit es a 1, significa que aquell node concret té una còpia de la línia.

## 2.3. Traduït als exàmens

Sempre apareix sota dos tipus de problemes matemàtics i lògics:

### Cas A: Càlcul del nombre de Bits de Coherència

Et donen l'arquitectura del sistema i demanaran calcular quants bits extra requereix el hardware.

#### 1. Bits en las caches (Snoopy / MSI):

- La regla: es necessiten sempre 2 bits per línia de cache.
- Exemple d'examen: un node té 4 caches de 4MB amb línies de 128 bytes.
  - Línies per cache:  $4 \text{ MB} / 128 \text{ B} = 2^{15}$  línies.
  - Bits per cache:  $2^{15} \text{ línies} * 2 \text{ bits} = 2^{16}$  bits totals

#### 2. Bits en el Directori (Memòria central / MSU):

- La regla: es necessiten  $2 \text{ bits d'estat} + N \text{ bits de presència}$  per a cada línia de memòria central, on N és el nombre de nodes NUMA.
- Exemple d'examen: 3 nodes NUMA, memòria de 8GB per node, línies de 32 bytes.
  - Línies en memòria:  $8 \text{ GB} / 32 \text{ bytes} = 2^{33} / 2^5 = 2^{28}$  línies
  - Bits per entrada: 2 (estat MSU) + 3 (un bit per cada node) = 5 bits
  - Total de bits al directori:  $5 * 2^{28}$  bits

### Cas B: Omplir taula de traces de Coherència (Timelines)

Et donen un codi seqüencial o paral·lel i una taula buida. S'ha de simular el comportament de las cache instrucció per instrucció. Aquestes són les regles a seguir:

#### 1. Lectura per primer cop (Read Miss)

- CPU command: PrRd (processor Read)
- Bus transaction: BusRd
- Estat cache local: passa a ser S
- Directori: l'estat passa a S i s'encén el bit de presència d'aquell node (ex: passa de 00 a 01)

#### 2. Lectura si un altre processador la tenia en M

- Si el processador 1 llegeix una variable que el processador 0 tenia en M, llavors processador 0 es veu forçat a fer flush als bus.
- Estat: la cache del processador 0 baixa de M a S, i la cache del processador 1 està en S. (en el Directori passa de M a S i s'encenen ambdós bits de presència)

#### 3. Escriptura per primer cop (Write Miss)

- CPU command: PrWr
- Bus transaction: BusRdX
- Estat cache: passa a ser M
- Efecte en els demés: si altres tenien la línia en S, s'invaliden (passen a I). Si algú tenia M, primer fa flush i després s'invalida (I).
- Directori: passa a estat M i només queda encés el bit de presència del node que acaba d'escriure.

#### 4. Escriptura si ja la tenia en S (Write Hit)

- CPU command: PrWr
- Bus transaction: BusUpgr (això és clau, no utilitzar BusRdX, perquè ja tens les dades, només vols pujar privilegis)
- Estat cache: puja de S a M. La resta de caches amb S passen a I.
- Directori: passa de S a M, i s'apaguen els bits de presència de tota la resta de nodes.

Consell: a vegades la trampa està en el False Sharing. Si el processador 0 escriu en A, després el processador 1 escriu en A, i si ambdós elements cauen a la mateixa línia de cache, el processador 1 generarà una fallada (Miss) amb BusRdX, forçant al processador 0 a fer flush i passar a I.

### 3. Tràfic de Coherència (True Sharing vs. False Sharing)

La **clau** d'aquest apartat radica en un concepte fonamental de l'arquitectura: la coherència de cache no es manté a nivell de variable o paraula, sinó a nivell de línia de cache completa.

#### 3.1. True Sharing

Sol succeir quan múltiples threads llegeixen i escriuen en la **mateixa variable exacta**. És inevitable perquè és l'essència de la programació paral·lela (per exemple, diversos threads acumulant un resultat sobre una variable *result*).

- Problema: per a evitar Race Conditions, es protegeix la variable amb un *#pragma omp atomic*, però al fer-ho, la línia de cache viatja constantment d'un processador a l'altre, generant molt de tràfic de coherència.
- Solució típica: creem variables temporals o locals (ex: `int tmp = 0`) on cada thread acumula el seu treball sense molestar a la resta, i només al final de tot es suma el resultat local al global.

#### 3.2. False Sharing

Sol succeir quan dos processadors accedeixen a **variables totalment distintes**, però, com estan emmagatzemades en posicions de memòria contigües, cauen dins de la **mateixa línia de cache**.

- Problema: si el processador 0 modifica la seva variable x i el processador 1 modifica la variable y, encara que lògicament no interfereixen, a nivell de hardware el sistema invalida la línia de cache sencera. Això genera efecte "ping-pong" on la línia viatja pel bus invalidant-se constantment, reduint el rendiment general.

#### 3.3. Traduït a exàmens

En els exàmens posen codis que funcionen perfectament a nivell lògic, però tenen un rendiment molt dolent. Hi ha tres escenaris possibles:

### Escenari A: Vectors o Estructures indexades per l'ID del thread

- Codi Trampa: declaren un array per a guardar resultats parcials de cada thread: `int sums[NUM_THREADS]`. Després cada thread fa `sums[my_id] += ...`.
- Anàlisis: com `num_threads` sol ser petit, tot el vector hi cap en una sola línia de cache. Tots els threads escriuen a la mateixa línia simultàneament, causant False Sharing.
- Solució (Padding 2D): afegir dimensions extra a l'array per a obligar a que la dada de cada thread ocupi una línia de cache sencera.
  - Resposta examen: declarar array com `int sums[NUM_THREADS][ELEMS_PER_CACHE_LINE]`. On `ELEMS_PER_CACHE_LINE` és la mida de la línia dividit per la mida de la dada. Així cada thread accedeix a `sums[my_id]`.

### Escenari B: Distribució Cíclica en Vectors (Cyclic Distribution)

- Codi Trampa: et donen un bucle paral·lel amb una distribució cíclica: `for (i = my_id; i < N; i += num_threads) A[i] = foo()`
- Anàlisis: el thread 0 procesa A, el thread 1 també, etc. Com cada posició està contigua a memòria, cauen a la mateixa cache line.
- Solució (Block-Cyclic): canviar la mida del chunk (bloc) per a que cada thread procesi un bloc equivalent a la mida exacte d'una línia de cache. Si la línia hi cap 16 enters, fas que cada thread procesi 16 enters seguits, evitant que threads comparteixin línia de cache.

### Escenari C: Matrius amb dimensions que no quadren

- Codi Trampa: et demanen procesar una matriu per files alternes (ex: core 0 fa files pars i core 1 les impars).
- Anàlisis: si la dimensió N de la matriu és petita o no és múltiple exacte de la mida de la línia de cache, el final de la fila par i el començament de la impar compartiran la mateixa memòria.
- Solució (Padding en Matrius/Structs): afegir un "relleno" o padding declarant la matriu amb columnes extra (que no s'utilitzaran) simplement per a forçar que la següent fila comenci en una línia de cache nova.
  - Resposta examen: en comptes de `double A[M][N]`, declarem `double A[M][N + PADDING]`, calculant el padding necessari per a omplir els bytes que falten fins a arribar al límit de la línia. Si és un struct, pots afegir explícitament `char padding` dins de l'estructura.

## 4. Suport Hardware per a la Sincronització

Una part molt enfocada en com l'arquitectura subjacent fa possibles implementacions de OpenMP que s'utilitza per a evitar Race Conditions.

Per a que directives com `#pragma omp atomic` o `#pragma omp critical` o funcions com `#pragma omp_set_lock()` funcionin, el processador ha de comptar amb instruccions indivisibles (atòmiques) a nivell de hardware. Una seqüència normal de "llegir, comprovar i escriure" (load-test-store) no és atòmica i provoca errors si diversos threads l'executen simultàneament.

Mecanismes teòrics i l'aplicació pràctica que surt a l'examen:

### 1. Primitives Bàsiques de Sincronització

L'arquitectura prové d'instruccions especials que fan l'escriptura i lectura en un únic pas atòmic a través del bus:

- Test-and-set (t&s): llegeix el valor d'una direcció de memòria i, simultàneament, escriu un 1 (o vertader) en ella.
- Atomic exchange (exch): intercanvia el valor d'un registre del processador amb un valor a memòria.
- Fetch-and-op: llegeix un valor i el reemplaça amb un resultat d'una operació aritmètica simple (com sumar, incrementar o decrementar). Aquesta és la instrucció que sol donar suport directe a `#pragma omp atomic`.

### 2. Optimització del Tràfic: "Test-test-and-test"

- El problema: si implementen un *lock* utilitzant un bucle que executa test-and-set contínuament fins aconseguir-ho, estarem generant escriptures constants. Això provoca tràfic massiu d'invalidacions de cache en el bus, baixant el rendiment.
- Solució teòrica: en classe s'explica la tècnica test-test-and-test. Consisteix en esperar primer una instrucció de lectura normal (load). Mentre el lock estigui ocupat, el thread es queda llegint la seva còpia neta en la cache (estat S, sense generar tràfic al bus). Només quan detecti que el lock s'ha alliberat (algú escriu un 0), llavors llença la instrucció atòmica test-and-set per a intentar adquirir-ho.

### 3. Parells d'instruccions (LL / SC) - El més important!

En sistemes moderns i escalables, implementar instruccions complexes en un sol pas és ineficient. L'arquitectura ho soluciona separant l'atomicitat en dos instruccions emparellades:

- Load-linked (LL): llegeix i retorna el valor actual d'una posició de memòria.
- Store-conditional (SC): intenta escriure un nou valor en aquella posició de memòria. La **clau** està en que l'escriptura només té èxit (retorna 1) si cap altre processador ha modificat aquella variable des de que va fer un LL. Si cap altre ho ha fet, retorna 0 i no escriu res.



## 4.1. Traduït als exàmens

L'exercici estrella d'aquest apartat és programar una funció de sincronització utilitzant únicament LL i SC.

**Exemple d'examen:** t'expliquen que disposes de primitives `ll(int *address)` i `int sc(int *address, int value)`. Et demanen implementar una funció com, *atomic\_inc*, que incrementi un comptador atòmicament, però respectant un tope màxim (saturació).

**Estructura de la resposta:** per a resoldre-ho sempre s'ha de posar LL i SC dins del bucle while que es repeteix mentre sc falli.

Exemple de codi:

```
C/C++
int atomic_inc(int *address, int max) {
    int tmp = ll(address); // Leemos el valor actual

    // Mientras no hayamos llegado al máximo y el intento de escritura (sc) falle
    while ((tmp < max) && !sc(address, tmp+1)) {
        tmp = ll(address); // Volvemos a leer si alguien más se adelantó
    }

    return tmp; // Devolvemos el valor antes de incrementar (o el máximo si saturó)
}
```

**Una altra variant:** a vegades a l'examen s'avalua la sincronització des de la perspectiva d'OpenMP explícit per a protegir estructures complexes. Per exemple, donen un array o graf i múltiples threads que han de modificar-ho de forma impredecible. Exigeixen inicialitzar un vector de locks (`omp_init_lock`), i protegir cada actualització utilitzant `omp_set_lock(&locks[pos])` i `omp_unset_lock(&locks[pos])` en comptes de posar *atomic*.

## 5. Implementació manual del Load Balancing

La base d'aquest apartat és la "Owner-Computes Rule". En sistemes de memòria distribuïda o NUMA, s'ha d'assignar el càlcul d'una dada a un thread el qual tindrà aquesta dada en la seva memòria local. S'han de fer implícit tasks, es a dir, només posar *#pragma omp parallel* i després manualment agafar la id de cada thread i calcular quantes iteracions farà cadascú.

Les tres estratègies més típiques:

### 5.1. Distribució geomètrica per Blocs (Block Data Decomposition)

Consisteix en dividir l'estructura de dades en blocs contigus i assignar un bloc a cada thread.

Per a repartir les iteracions cal fer servir la plantilla següent per a garantir un bon load balancing:

```
C/C++
#pragma omp parallel
{
    int id = omp_get_thread_num();
    int nths = omp_get_num_threads();

    int BS = N / nths;          // mida del bloc
    int mod = N % nths;         // elements sobrants

    int start = id * BS;
    int end = start + BS;

    // repartim iteracions sobrants
    if (mod > 0) {
        if (id < mod) {
            start += id;          // desplaçament threads anteriors
            end = start + BS + 1; // farà una iteració extra
        } else {
            start += mod;
            end += mod;           // afegim desplaçament extra
        }
    }

    for (int i = start; i < end; i++) {
        // càlcul
    }
}
```

## 5.2. Distribució Cíclica (Cyclic Data Decomposition)

En aquesta estratègia cada thread fa la iteració amb el seu id de thread i després passa a la següent sumant la quantitat de threads totals. L'avantatge respecte l'anterior es que no cal utilitzar càlculs de mod ni res.

```
C/C++
#pragma omp parallel
{
    int id = omp_get_thread_num();
    int n_ths = omp_get_num_threads();

    for (int i = id; i < N; i += n_ths) {
        // ... v[i] ...
    }
}
```

A l'examen poden canviar com calcular l'index dins del bucle (index = func(i)), llavors s'hauria de fer la comprovació if (index % n\_ths == id).

## 5.3. Distribució Bloc-Cíclica (Block-Cyclic Data Decomposition)

Aquesta és una mescla de totes dues anterior. Consisteix en repartir blocs contigus de mida fixa (BS), de forma cíclica entre els threads.

Es sol demanar per a minimitzar el **tràfic de coherència**. Si assignem a cada thread un bloc (BS) equivalent a les dades exactes que hi caben en una línia de cache, llavors evitem False Sharing i maximitzem localitat espacial, i alhora tenim un bon load balance. El desbalanceig màxim només pot ser de BS - 1 elements.

```
C/C++
#pragma omp parallel
{
    int id = omp_get_thread_num();
    int nths = omp_get_num_threads();
    int BS = CACHE_LINE_SIZE / sizeof(int); // bloc adaptat a la cache

    // salt cíclic
    for (int ii = id * BS; ii < N; ii += nths * BS) {
        // processat bloc intern
        for (int i = ii; i < min(N, ii + BS); i++) {
            // ... v[i] ...
        }
    }
}
```

## 6. Quina Descomposició Geomètrica de Dades escollir

Aquest apartat es centra en com escollir la Descomposició Geomètrica de Dades més adequada (per blocs, cíclica o bloc-cyclic, per files o per columnes) basant-se en les dependències de dades del codi seqüencial.

En els exàmens, donen un bucle anidat que actualitza una matriu, i demanaran escriure la versió paral·lela en OpenMP justificant l'elecció. Per a obtenir la màxima puntuació s'han de complir 3 objectius: minimitzar la sincronització, equilibrar càrrega i maximitzar localitat de les dades.

Pas a pas com resoldre un exercici d'aquest tipus:

### 1. Anàlisis de Dependències (pas crític)

El primer a mirar són els índexs dels accessos a la matriu dins dels bucles per a identificar dependències vertaderes (RAW) o falses (WAR). L'objectiu és escollir la dimensió (files o columnes) que **no tinguin dependències**, ja que això permet que cada thread treballi de forma individual.

- **Exemple:** et donen un càlcul del tipus  $m[i][j] = \text{compute}(m[i][j], m[i-1][j], m[i+1][j])$
- **Raonament exigít:** per a calcular un element de la fila  $i$  cal accedir a la fila anterior ( $i-1$ ) i a la següent ( $i+1$ ). Però, pel que fa a les columnes només necessita la  $j$ , per tant, no hi ha dependències sobre les columnes.
- **Solució:** s'ha de proposar explícitament una descomposició geomètrica per columnes, de mode que cada thread se li assigni un conjunt de columnes completes i ningú interfereixi.

### 2. Selecció de patró: Block, Block-Cyclic, Cyclic

Una vegada escollida la dimensió a dividir (per exemple, per columnes), hem de decidir com repartir les columnes entre els threads per a tindre un bon load balance.

- **Distribució per Blocs (Block):** és la millor opció si el treball que requereix cada cel·la de la matriu és uniforme. En l'examen cal utilitzar el codi vist anteriorment.
- **Distribució Cíclica (Cyclic):** si el codi té bucles triangulars o irregulars (ex: `for (int j = i; j < N; ++j)`) el volum de còmput no és uniforme, ja que les primeres iteracions i processen molts més elements que les finals. Per tant, cal utilitzar cyclic per a obtenir un bon load balance.

### 3. Maximitzar la Localitat i evitar False Sharing

En el llenguatge C, les matrius bidimensionals s'emmagatzemen seqüencialment en memòria per files (Row-wise storage). Si en el pas 1 s'ha decidit descomposar per columnes per a evitar problemes de sincronització, llavors perdem localitat espacial, ja que els threads accedeixen a posicions no contigües a memòria.

Als problemes indicaran clàrament la mida de la línia de cache (ex: 128 B) i la mida de la dada (ex: enters de 4 B).

- **Block-Cyclic ajustat a la cache:** per a no penalitzar la localitat al repartir per columnes llavors s'ha de proposar fer una descomposició Bloc-Cyclic per columnes, on la mida del bloc (BS) sigui igual al nombre d'elements que hi caben a una línia de cache ( $128 / 4 = 32$  enters).

- **Afegir Padding:** si l'ample original de la matriu N no resulta ser un múltiple exacte dels bytes de la línia de cache, llavors hi haurà False Sharing i molt tràfic. Per a corregir això cal justificar i declarar una estructura de Padding (columnes buides per omplir), per exemple, declarant la matriu com `m[N][N + PADDING]` calculant matemàticament els forats extres fins alinear perfectament amb la línia de cache.

## **7. Ajust de la Línia de Cache per a evitar False Sharing**

En els exàmens, aquest apartat s'avalua plantejant problemes on la distribució geomètrica (per blocs o cíclica) genera ineficiències greus degut a la mida de la línia de cache. Exigiran modificar el codi dels bucles com la declaració de les estructures de dades.

Tres estratègies a seguir:

- **Descomposició Bloc-Cyclic orientada a la Cache (Block-Cyclic)**

Si demanen paral·lelitzar un codi, et donen la mida de la línia de cache i també la mida de la dada, l'objectiu és assignar a cada thread blocs de memòria que coincideixen exactament amb la mida de la línia de cache.

- Càlcul inicial obligatori: sempre hem de calcular el nombre d'elements que hi caben a una línia de cache.
- Implementació del bucle: en comptes de repartir de forma cíclica element a element, hem de programar una distribució Bloc-Cíclica. El bucle exterior donarà salts grans multiplicant la mida del bloc pel nombre de threads ( $BS * n\_ths$ ), i el bucle interior processa els elements d'aquell bloc (BS).
- Exemple d'examen: demanen crear histograma d'un vector gegant. Per a evitar False Sharing, es divideix el domini dels bins de l'histograma calculant  $BS = \text{CACHE\_LINE\_SIZE} / \text{sizeof}(\text{int})$  i s'assigna el treball verificant si el bin pertany al bloc del thread actual mitjançant  $(\text{bin}/BS) \% n\_ths == \text{id}$ .

- **Padding en matrius 2D**

En el llenguatge C les files es guarden de forma consecutiva en memòria, i per tant, quan l'ample de la fila N no és múltiple exacte dels elements que hi caben en una fila hi ha problemes. Si ocorreix això, el final de la fila i i el principi de la fila i + 1 coincideixen a la mateixa línia de cache.

- El que demanen: modificar la declaració de la matriu per afegir Padding (columnes falses de relleno al final de cada fila) de mode que cada fila nova comença perfectament amb una línia de cache nova.
- Fòrmula:  $\text{PADDING} = (\text{CACHE\_LINE\_SIZE} - (N * \text{sizeof}(\text{double}) \% \text{CACHE\_LINE\_SIZE})) / \text{sizeof}(\text{double})$

- **Padding en Estructures i Vectors d'Acumuladors**

En molts codis, es declara un vector petit indexat pel id del thread o estructures on diversos threads actualitzen diferents camps (ex: `int sum[NUM_THREADS]`). Com els elements són contigus, tots els threads intenten escriure sobre la mateixa línia de cache.

Per a solucionar-ho hi ha dos escenaris:

- Vectors indexats per thread (Padding 2D): si tens un vector global com `int sums[NUM_THREADS]` per a guardar el resultat parcial de cada thread, cal declarar-ho un altre cop afegint una segona dimensió que actuï de separació.
- Estructures (Padding intern): si tens un array d'estructures on cada thread accedeix a una posició, s'ha de ficar un camp fals de tipus `char` dins de l'estruct per a engreixar la mida del mateix fins que coincideix amb la línia de cache.

## **8. Suport Hardware per a la Sincronització**

Aquest apartat es centra en la dicotomia entre Input Data Decomposition i Output Data Decomposition aplicant la regla "Owner Computes Rule" (el thread al qual se li assigna unes dades les ha de calcular).

En els exàmens sempre surt com un problema de càlcul d'un histograma (o conteig de quantes vegades surt un element). Tens un vector d'entrada molt gran (les dades) i un vector de sortida més petit (els bins o comptadors del histograma).

Exigeixen paral·lelitzar el mateix codi de dues formes diferents per a contrastar els seus avantatges i inconvenients. S'ha de plantejar de la següent forma:

### **8.1. Output Data Decomposition**

Aquesta estratègia es basa en dividir l'estructura de sortida (vector de l'histograma) entre els threads disponibles. Cada thread es converteix en el propietari exclusiu d'un rang de bins de l'histograma.

- Com es programa a l'examen: tots els threads recorren el vector d'entrada de principi a fi, però només un thread incrementa el comptador si el valor que acaba de llegir cau dins dels bins que hi té assignat.
- Avantatge a destacar: com cada thread és propietari de posicions distintes de la sortida, no hi ha Race Conditions i no es requereix cap sincronització com `#pragma omp atomic`

### **8.2. Input Data Decomposition**

Aquí fem el contrari, dividim el vector massiu d'entrada en blocs (ex:  $\text{MAXELEM} / P$ ) i se li dona als threads.

- Problema de l'examen: ara cada thread processa exclusivament el seu tros d'entrada, però tots els threads poden acabar intentant actualitzar la mateixa posició del vector global de sortida al mateix temps, lo qual corromperia les dades.
- Solució:
  - Ús d'histogrames locals (Replicació): per a intentar evitar omplir el bus de memòria amb centenars de `#pragma omp atomic`, la solució oficial esperada es que cada thread creï un histograma local i privat inicialitzat a 0. El thread compta les seves dades localment sense bloqueigs. Una vegada termina el seu bloc, posa el resultat en l'histograma global mitjançant un bucle protegit.

- Ús de locks per a estructures complexes: si l'actualització de l'histograma implica diversos camps a la vegada (ex: un struct amb un comptador i una suma total), un atomic no és suficient. S'ha de crear un vector de locks (`omp_lock_t locks[P]`) i inicialitzar-los. A l'actualitzar, el thread fa `omp_set_lock(&locks[pos])`, realitza una escriptura complexa i el llibera amb `omp_unset_lock(&locks[pos])`.